High Performance Computing with C++

LSEG Technology

12th March 2026

LSEG

Where do we see multi-threading?

- Text File Editor/ MS Word
  - Respond to keyboard
  - Spell Check
  - Formatting
- Computer game
  - Different objects rendered as different threads
- Media Player
  - Run a song/Video
  - List down the playlist
  - Update the playlist
  - Searching a song feature
- Web Browser
  - Multiple tabs working simultaneously
  - Downloading files

Multithreading in C++

- Threads allow multiple functions to execute concurrently
- Share same address space
- Address performance concerns
- Do tasks in parallel
- Before C++11, pthread. with C++11 std::thread

std::thread

- Represents a single thread in C++
- A function pointer
- A function object
- A lambda expression

std::thread – Function pointers

```cpp
#include <iostream>
#include <thread>

using namespace std::chrono_literals;

void rest_api_call1(const std::string& endpoint)
{
    //Using for loop and sleep to indicate long running functions
    for (int i = 0; i < 10; i++)
    {
        std::cout << "Calling rest_api_1 [" << endpoint.c_str() << "] ..." << std::endl;
        std::this_thread::sleep_for(1s);
    }
}

void rest_api_call2(const std::string& endpoint)
{
    //Using for loop and sleep to indicate long running functions
    for (int i = 0; i < 5; i++)
    {
        std::cout << "Calling rest_api_2 [" << endpoint.c_str() << "] ..." << std::endl;
        std::this_thread::sleep_for(3s);
    }
}

int main()
{
    std::thread restCall1(rest_api_call1, "endpoint1");
    std::thread restCall2(rest_api_call2, "endpoint2");

    restCall1.join();
    restCall2.join();

    std::cout << "All threads completed" << std::endl;

    std::cin.get();
}
```

std::thread – Function object

```cpp
#include <iostream>
#include <thread>

using namespace std::chrono_literals;

class RestApiCall1 {
public:
    void operator()(const std::string endpoint)
    {
        //Using for loop and sleep to indicate long running functions
        for (int i = 0; i < 10; i++)
        {
            std::cout << "Calling rest_api_1 [" << endpoint.c_str() << "] ..." << std::endl;
            std::this_thread::sleep_for(1s);
        }
    }
};

class RestApiCall2 {
public:
    void operator()(const std::string endpoint)
    {
        //Using for loop and sleep to indicate long running functions
        for (int i = 0; i < 5; i++)
        {
            std::cout << "Calling rest_api_2 [" << endpoint.c_str() << "] ..." << std::endl;
            std::this_thread::sleep_for(3s);
        }
    }
};

int main()
{
    std::thread restCall1(RestApiCall1(), "endpoint1");
    std::thread restCall2(RestApiCall2(), "endpoint2");

    restCall1.join();
    restCall2.join();

    std::cout << "All threads completed" << std::endl;

    std::cin.get();
}
```

LSEG

std::thread – Lambda expression

```
2   #include <iostream>
3   #include <thread>
4
5   using namespace std::chrono_literals;
6
7   int main()
8   {
9       auto f1 = [](const std::string endpoint) {
10          for (int i = 0; i < 10; i++)
11          {
12              std::cout << "Calling rest_api_1 [" << endpoint.c_str() << "] ..." << std::endl;
13              std::this_thread::sleep_for(1s);
14          }
15      };
16
17      auto f2 = [](const std::string endpoint){
18          for (int i = 0; i < 5; i++)
19          {
20              std::cout << "Calling rest_api_2 [" << endpoint.c_str() << "] ..." << std::endl;
21              std::this_thread::sleep_for(3s);
22          }
23      };
24
25      std::thread restCall1(f1, "endpoint1");
26      std::thread restCall2(f2, "endpoint2");
27
28      restCall1.join();
29      restCall2.join();
30
31      std::cout << "All threads completed" << std::endl;
32
33      std::cin.get();
34  }
35
```

std::thread – Aborts without join/detach

When a std::thread object is destroyed while it is still joinable (i.e., the underlying thread hasn't been join()ed or detach()ed), the C++ runtime calls std::terminate(), which aborts your program.

In Linux

```
terminate called without an active exception
Aborted (core dumped)
```

or on MSVC:

```
std::terminate called
```

What does "joinable" mean?

Right after you start a thread, t.joinable() returns true.

After you call t.join() or t.detach(), t.joinable() becomes false.

If the thread object reaches the end of its lifetime while it is still joinable, destruction triggers std::terminate().

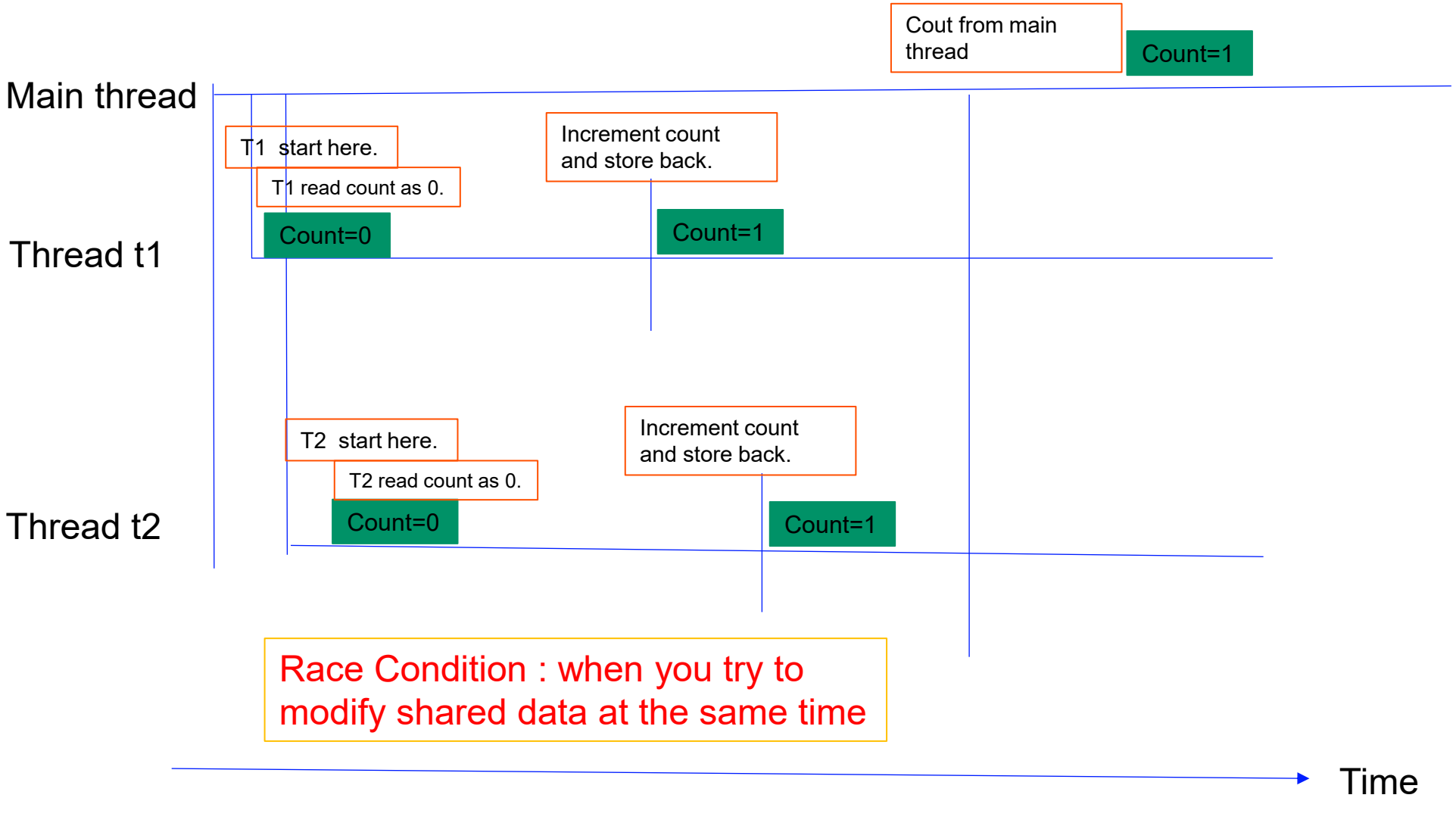
Why? prevents silent bugs where a thread keeps running after its owning object disappears, like use-after-free, data races

std::jthread – C++20

Auto-joining (RAII) – Safer threading
Built-in cooperative cancellation (stop tokens)

Shared Data Between Threads

```
int main()
{
    int count = 0;


    thread t1([&count](){
            count++;
    });
    thread t2([&count](){
            count++;
    });

    t1.join();
    t2.join();

    cout<<"End"<<count<<endl;
}
```



What happens behind count ++

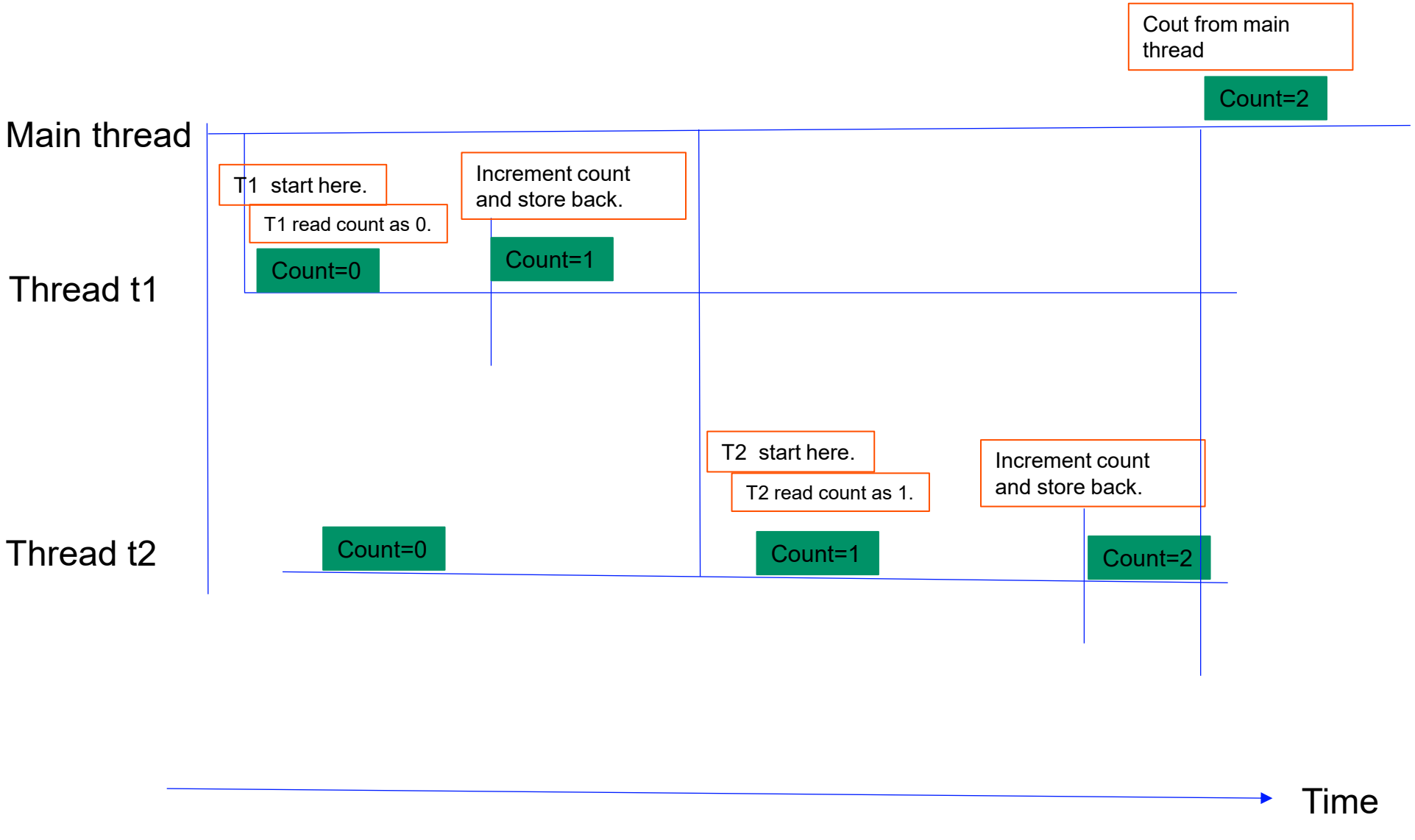
```
int main()
{
    int count = 0;


    thread t1([&count](){
            count++;
    });
    thread t2([&count](){
            count++;
    });

    t1.join();
    t2.join();

    cout<<"End"<<count<<endl;
}
```

Not an atomic operation!!

Count variable copied to a CPU register → Increment it there → Save it back to memory

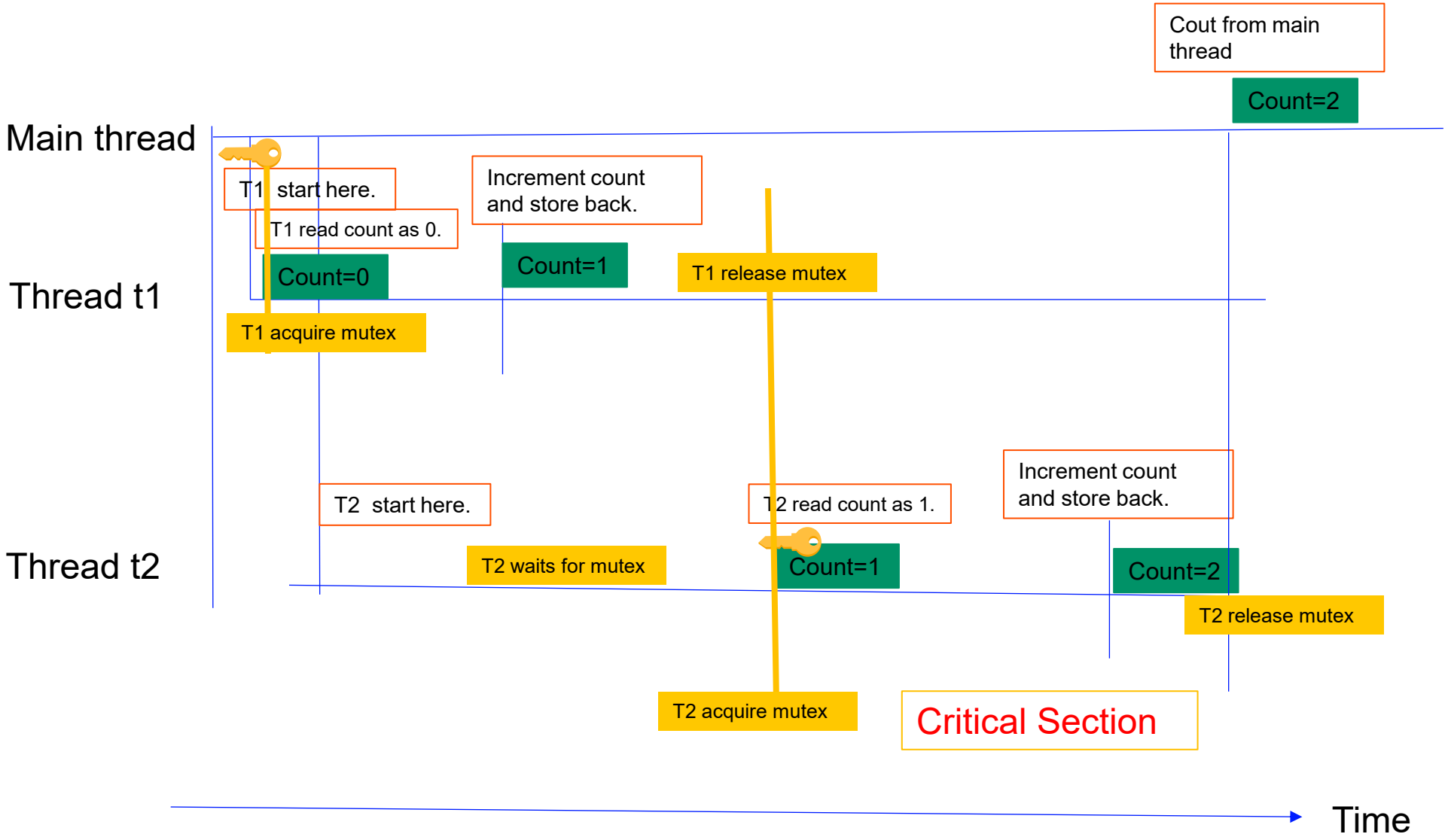
Shared Data Between Threads

```
int main()
{
    int count = 0;


    thread t1([&count](){
            count++;
    });
    thread t2([&count](){
            count++;
    });

    t1.join();
    t2.join();

    cout<<"End"<<count<<endl;
}
```



Mutex – a key to access shared data

```
#include <mutex>

using namespace std;

int main()
{
    int count = 0;
    mutex mtx;

    thread t1([&](){
        mtx.lock();
        count++;
        mtx.unlock();
    });
    thread t2([&](){
        mtx.lock();
        count++;
        mtx.unlock();
    });

    t1.join();
    t2.join();

    cout<<"End"<<count<<endl;
}
```



Atomic vs Mutex

Feature	std::atomic	std::mutex
Implemented by	CPU instructions	OS kernel
Context switch?	No	Often
Fast?	Very fast	Slower
Useful for	simple values	complex data

Mutex

```
#include <mutex>

using namespace std;

int main()
{
    int count = 0;
    mutex mtx;

    thread t1([&](){
            mtx.lock();
            count++;
            mtx.unlock();
    });
    thread t2([&](){
        mtx.lock();
            count++;
            mtx.unlock();
    });

    t1.join();
    t2.join();

    cout<<"End"<<count<<endl;
}
```

#include <mutex>

To protect shared data from being simultaneously accessed by multiple threads.

Need to make sure we unlock()

https://en.cppreference.com/w/cpp/thread/mutex

Mutex : try_lock()

```
int count = 0;

mutex mtx;

auto func = [&](){
    for (size_t i = 0; i < 100000; i++)
    {
        if(mtx.try_lock())
        {
            count++;
            mtx.unlock();
        }
    }
};
```

#include <mutex>

Try to get the lock

- If it gets lock: return back with true
- If it failed to get lock: return back with false

Anyway, returns back immediately !

If lock acquired, need to call unlock() to release

May return false even if it is not owned by any other thread.

If try_lock is called by a thread that already owns the mutex, the behavior is undefined.

https://en.cppreference.com/w/cpp/thread/mutex/try_lock

Timed Mutex : try_lock_for() or try_lock_until()

```
timed_mutex mtx;

auto func = [&](){
    for (size_t i = 0; i < 100000; i++)
    {
        //if(mtx.try_lock_for(chrono::milliseconds(5)))
        if(mtx.try_lock_until(chrono::steady_clock::now() + chrono::seconds(10)))
        {
            count++;
            mtx.unlock();
        }
    }
};
```

#include <mutex>

Similar to mutex

But can try to get the lock for a given time

- If it gets lock: return back with true
- If it failed to get lock: return back with false

Try_lock_for : for a given time duration, eg: for 5s
Try_lock_until : until a specified time point, eg: 11:50:50 th second

https://en.cppreference.com/w/cpp/thread/timed_mutex

GREAT IF WE HAVE TIME CONSTRAINTS IN RUNNING THE PROGRAM

Recursive Mutex

```
using namespace std;


class RecursiveClass {
    recursive_mutex m;
    string shared;
  public:
    void func1() {
      m.lock();
      cout<< "function1" <<endl;
      m.unlock();
    }

    void func2() {
      m.lock();
      func1();
      cout << "function 2" <<endl;
      m.unlock();
    };
};

int main()
{
    RecursiveClass rc;
    thread t1(&RecursiveClass::func1, &rc);
    thread t2(&RecursiveClass::func2, &rc);
    t1.join();
    t2.join();
}
```

#include <mutex>

Similar to mutex + Provide recursive ownership

The thread which currently owns lock can call lock() recursively.

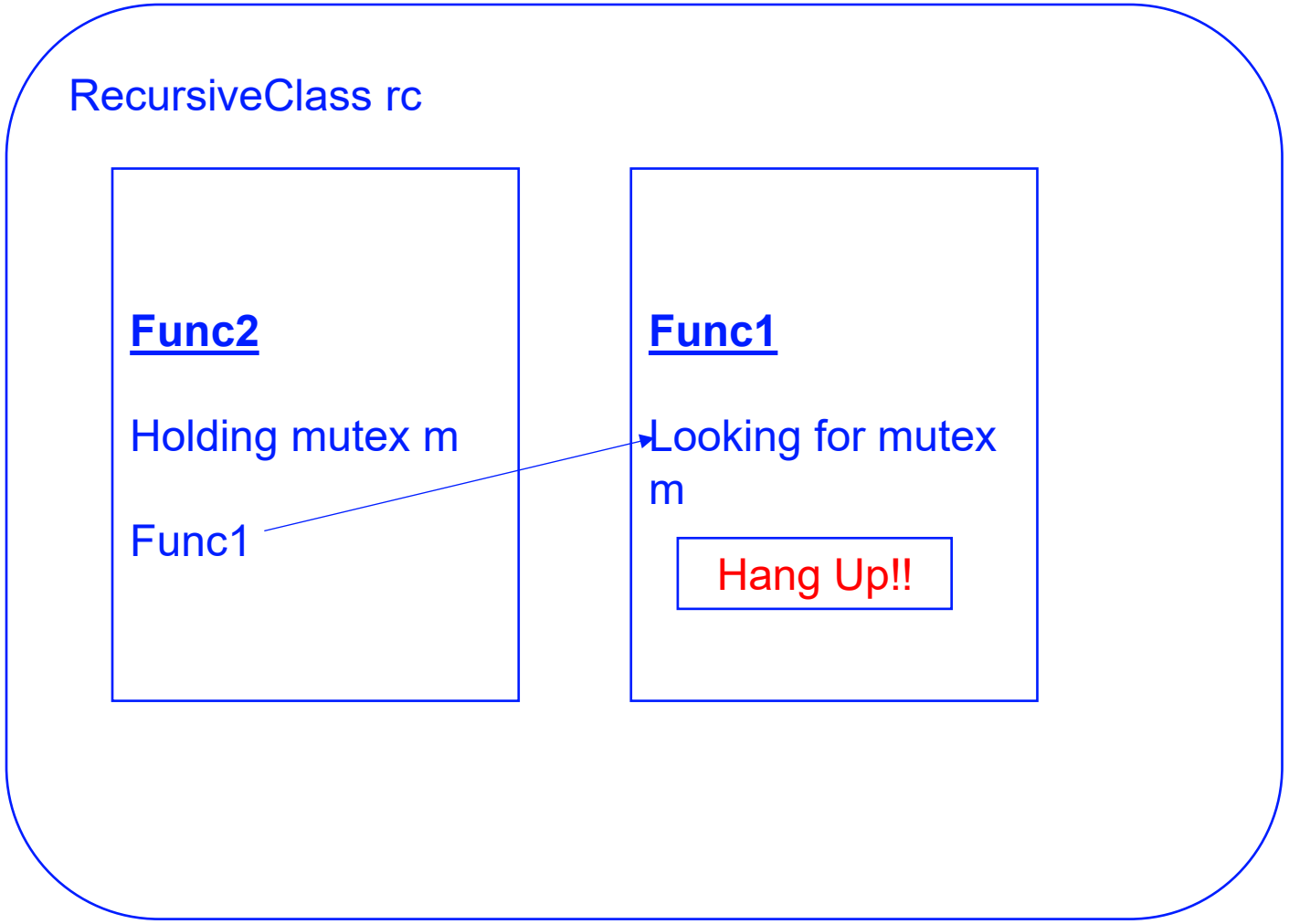
See recursive_timed_mutex too

https://en.cppreference.com/w/cpp/thread/recursive_mutex

It should unlock() for the same number of times it acquired lock()/try_lock()

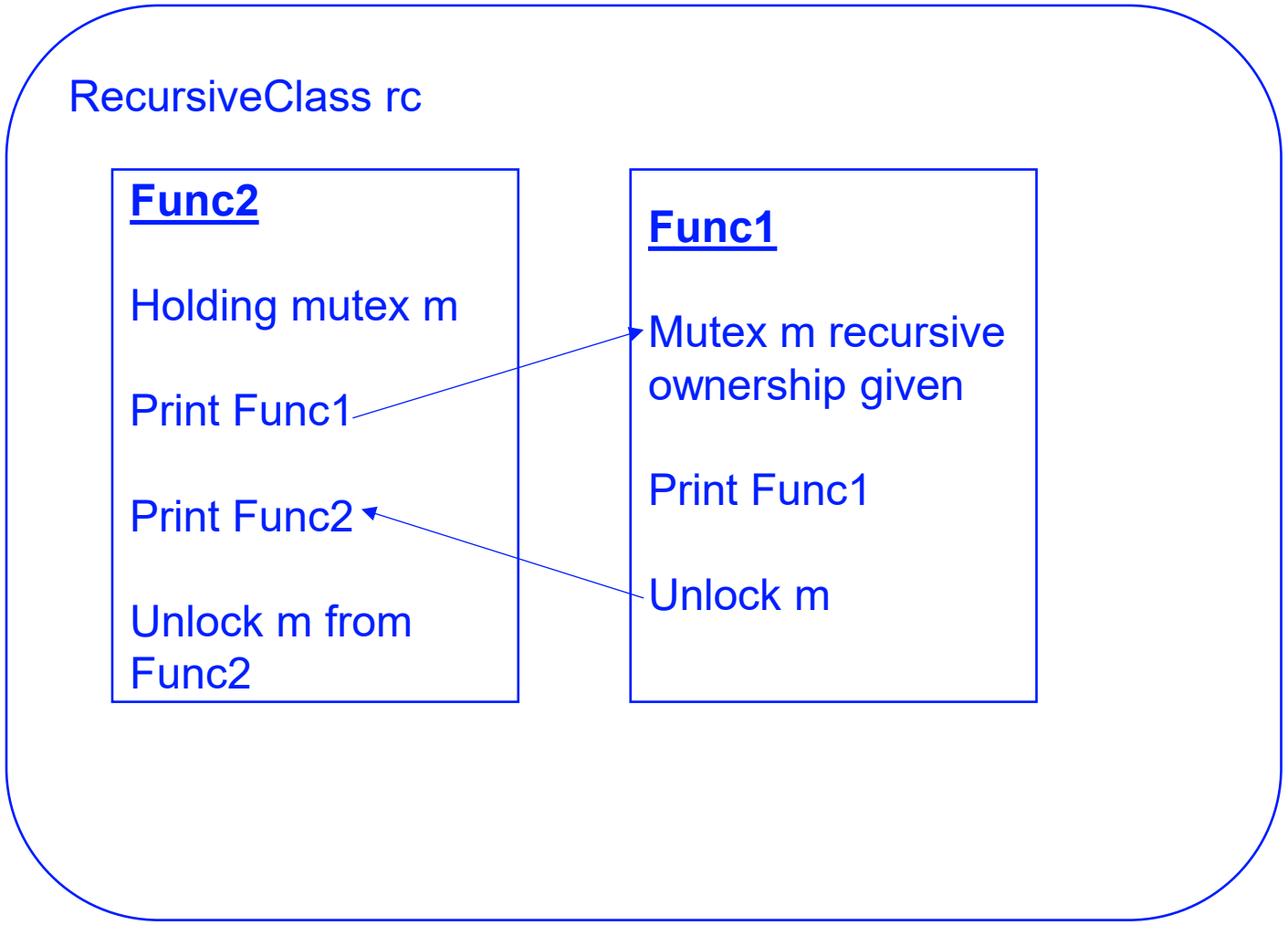
Recursive Mutex

```
using namespace std;


class RecursiveClass {
    recursive_mutex m;
    string shared;
  public:
    void func1() {
      m.lock();
      cout<< "function1" <<endl;
      m.unlock();
    }

    void func2() {
      m.lock();
      func1();
      cout << "function 2" <<endl;
      m.unlock();
    };
};

int main()
{
    RecursiveClass rc;
    thread t1(&RecursiveClass::func1, &rc);
    thread t2(&RecursiveClass::func2, &rc);
    t1.join();
    t2.join();
}
```



Recursive Mutex

```
using namespace std;


class RecursiveClass {
    recursive_mutex m;
    string shared;
  public:
    void func1() {
      m.lock();
      cout<< "function1" <<endl;
      m.unlock();
    }

    void func2() {
      m.lock();
      func1();
      cout << "function 2" <<endl;
      m.unlock();
    };
};

int main()
{
    RecursiveClass rc;
    thread t1(&RecursiveClass::func1, &rc);
    thread t2(&RecursiveClass::func2, &rc);
    t1.join();
    t2.join();
}
```



Shared Mutex (C++17)

```
class Counter {
private:
  shared_mutex mtx;
  int count = 0;
public:

  int get() {
    mtx.lock_shared();
    int i = count;
    mtx.unlock_shared();

    return i;
  }
```

#include <shared_mutex>

Similar to mutex + Provide shared access

Read Write Lock Behavior

2 Levels of access:

- shared: several threads can share ownership
  - Possible if no other threads have taken an exclusive lock
  - lock_shared(), try_lock_shared(), unlock_shared
- exclusive: only 1 thread can own the mutex
  - lock(), try_lock(), unlock()

If 1 thread has taken a shared lock, other threads too can take shared lock. But, not exclusive lock

https://en.cppreference.com/w/cpp/thread/shared_mutex

Lock_guard

```
void methodA(int& count, mutex& mtx)
{
    for (size_t i = 0; i < 100000; i++)
    {
        lock_guard<mutex> guard(mtx);
        //mtx.lock();
        count++;
        //mtx.unlock();
    }
}
```

#include <mutex>

RAII mechanism : Resource Acquisition is Initialization

Light weight wrapper around mutex which make sure RAII

When going out of scope,
Lock guard releases the mutex it is owning.

Non-copyable

Scoped_lock : from c++17 onwards, can use with several mutexes.

https://en.cppreference.com/w/cpp/thread/lock_guard
https://en.cppreference.com/w/cpp/thread/scoped_lock

Unique_lock

```
void methodA(int& count, mutex& mtx)
{
    for (size_t i = 0; i < 100000; i++)
    {
        unique_lock<mutex> guard(mtx);
       // mtx.lock();
        count++;
        //mtx.unlock();
       guard.unlock();


    }
}
```

#include <mutex>

Light weight wrapper around mutex which make sure RAII

+ Allow
  - ✓ deferred locking(lock later in code)
  - ✓ time-constrained attempts at locking
  - ✓ recursive locking
  - ✓ transfer of lock ownership
  - ✓ use with condition variables

When going out of scope,
releases the mutex if it is owning still.

Non-copyable

Scoped_lock : from c++17 onwards, can use with several mutexes.

https://en.cppreference.com/w/cpp/thread/lock_guard
https://en.cppreference.com/w/cpp/thread/scoped_lock

Deadlock

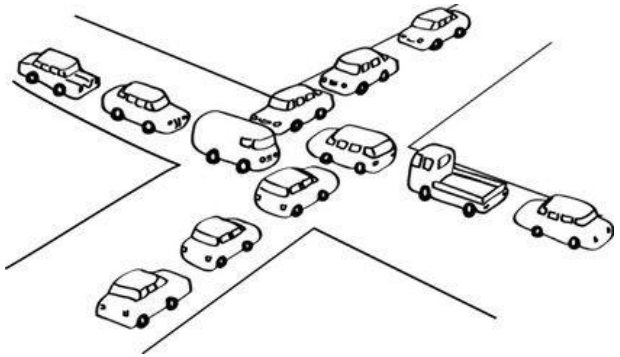
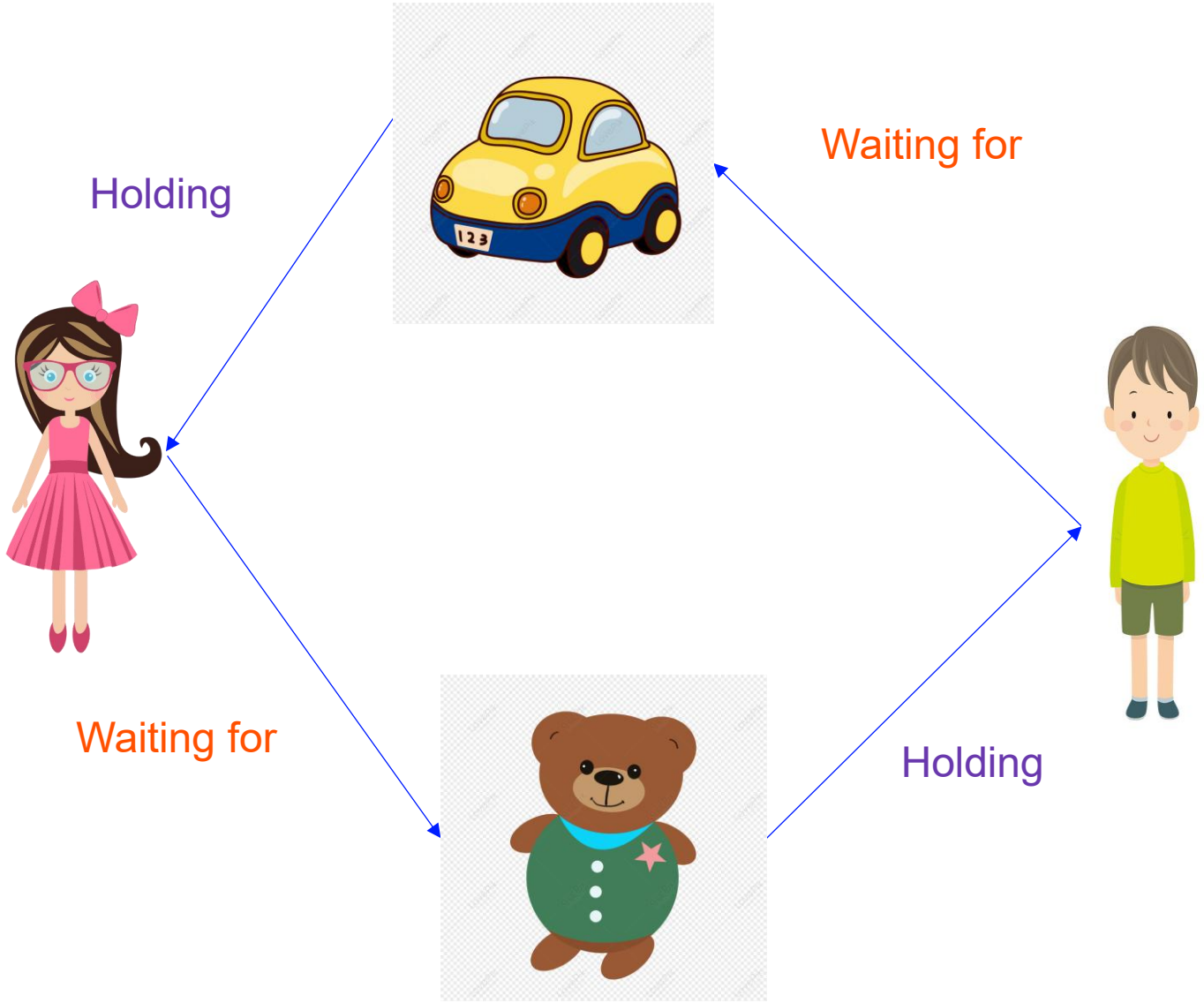


Image courtesy: https://arctype.com/blog/database-deadlock/



Deadlock

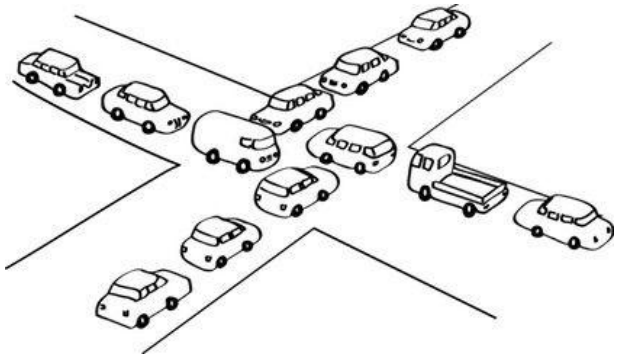
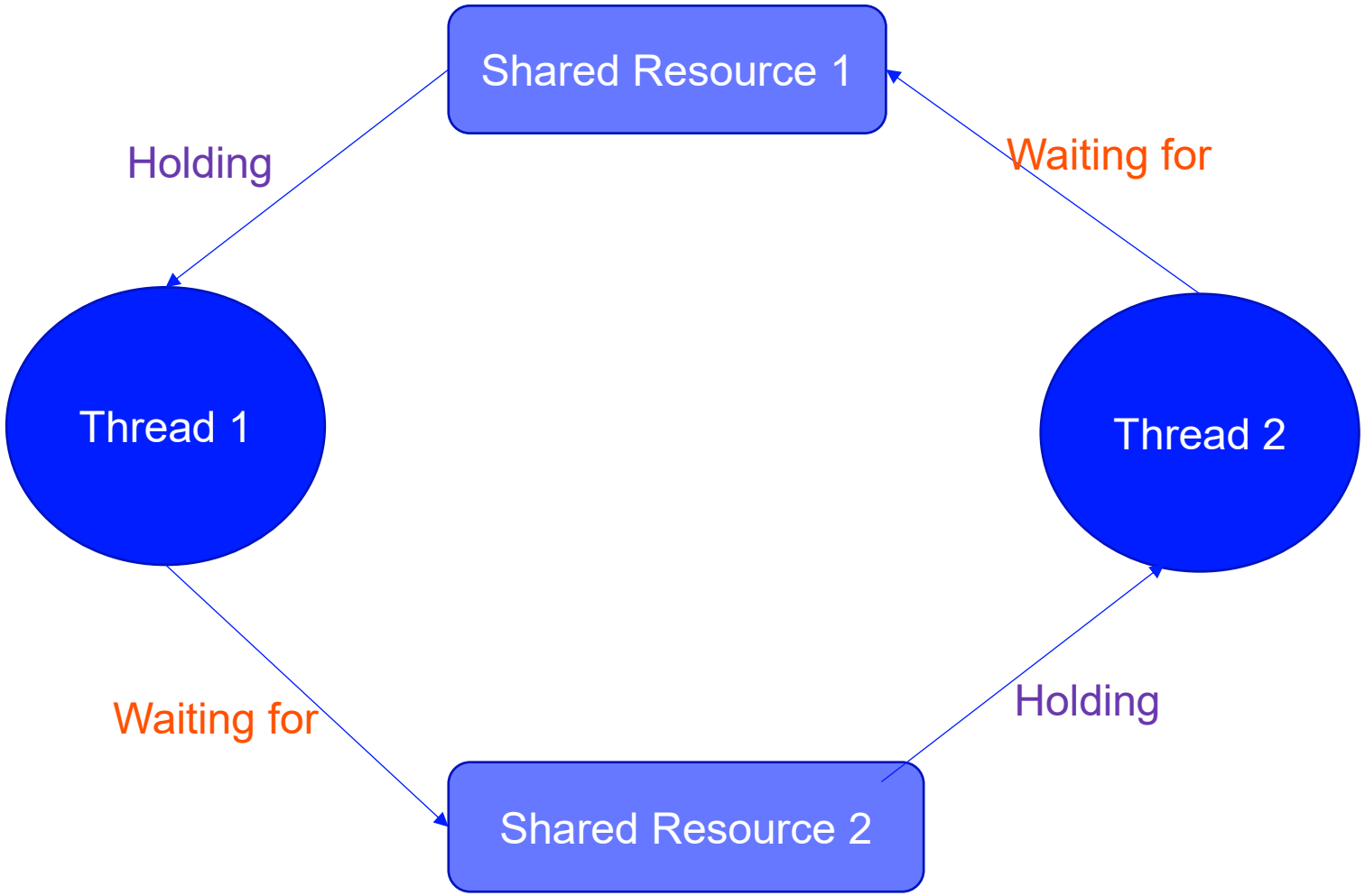


Image courtesy: https://arctype.com/blog/database-deadlock/



Mutexes prevent races, but they do not let threads communicate…

```
while (!ready) {
    // BAD: this is busy-waiting
}
```

Imagine Thread A is waiting for Thread B to finish some work.
With only mutexes, A must keep checking again and again - wasting CPU.

Thread A wastes CPU by spinning
Thread B sets ready = true eventually
Mutexes can protect ready, but cannot block a thread waiting for it
We need *a mechanism to sleep until a condition changes*

Condition Variable

```
int main()
{
    int i =0;
    mutex mtx;
    condition_variable cv;

    thread t1([&](){
        this_thread::sleep_for(chrono::seconds(1));

        unique_lock<mutex> lock(mtx);

        i++;
        lock.unlock();

        cv.notify_one();
    });

    t1.join();

    unique_lock<mutex> lock(mtx);

    cv.wait(lock, [&](){ return i ==1;});


    cout << "Value : "<<i<<endl;
}
```

#include <condition_variable>

Allow multiple threads to communicate with each other

Wait for 1/ more threads until 1 thread notifies

Always associated with a mutex

https://en.cppreference.com/w/cpp/thread/condition_variable

Semaphores | Latches | Barriers (C++20)

Semaphores:

#include <semaphore>

To constrain concurrent access to a shared resource

Counting_semaphore : a non-negative resource count

Binary_semaphore: only 2 states

https://en.cppreference.com/w/cpp/thread/counting_semaphore

Latches:

#include <latch>

Coorination mechanism to block until a given number of threads arrive at a given stage

Cannot reuse (cannot reset the counter)

https://en.cppreference.com/w/cpp/thread/latch

Barriers:

#include <barrier>

Coorination mechanism to block until a given number of threads arrive at a given stage

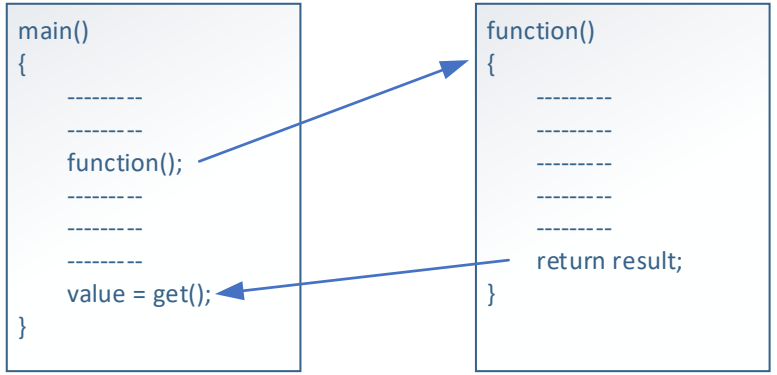
Can resue

https://en.cppreference.com/w/cpp/thread/barrier

How do I get a result back from a thread?

std::future and std::promise

- Provides a mechanism to access the result of asynchronous operations
- std::promise
- use to set values or exceptions
- std::future
- used to get value from promise
- wait for the promise



std::future and std::promise

```cpp
#include <iostream>
#include <future>
#include <thread>

void calculate(std::promise<uint64_t>&& prom, uint64_t from, uint64_t to)
{
    uint64_t sum = 0;
    std::cout << "Thread id of calculate: " << std::this_thread::get_id() << std::endl;
    for (uint64_t i = from; i < to; i++)
    {
        if (i & 1)
        {
            sum += i;
        }
    }
    prom.set_value(sum);
}

int main()
{
    std::cout << "Thread id of main (Caller): " << std::this_thread::get_id() << std::endl;

    uint64_t from = 0;
    uint64_t to = 7000000000;
    std::promise<uint64_t> prom;
    std::future<uint64_t> fut = prom.get_future();

    std::thread worker(calculate, std::move(prom), from, to);

    std::cout << "Waiting for results ..." << std::endl;
    std::cout << "Result: " << fut.get() << std::endl;

    std::cout << "Completed!" << std::endl;
    worker.join();

    std::cin.get();
}
```

std::async

- Runs a function asynchronously and returns std::future
- Launch policies
- std::launch::async
- std::launch::deferred
- std::launch::async | std::launch::deferred
- Automatically creates a thread or take from internal pool and create std::promise object
- Pass the std::promis object to thread and return std::future object

std::async

```
2   #include <iostream>
3   #include <future>
4
5   uint64_t calculate(uint64_t from, uint64_t to)
6   {
7       uint64_t sum = 0;
8       std::cout << "Thread id of calculate: " << std::this_thread::get_id() << std::endl;
9       for (uint64_t i = from; i < to; i++)
10      {
11          if (i & 1)
12          {
13              sum += i;
14          }
15      }
16      return sum;
17  }
18
19  int main()
20  {
21      std::cout << "Thread id of main (Caller): " << std::this_thread::get_id() << std::endl;
22      uint64_t from = 0;
23      uint64_t to = 7000000000;
24      std::future<uint64_t> result = std::async(std::launch::async, calculate, from, to);
25
26      std::cout << "Waiting for results ..." << std::endl;
27      std::cout << "Result: " << result.get() << std::endl;
28
29      std::cout << "Completed!" << std::endl;
30
31      std::cin.get();
32  }
33
```

Thread Binding

- Constraining a thread to run only on a specific CPU (or a set of CPUs).
- By default, the OS scheduler can run your thread on any core.
- Binding can help when you want:
  - **Cache locality:** keep a thread on the same core as the data it frequently touches (reduces cache misses).
  - **Determinism & jitter reduction**: real-time-ish workloads, low-latency trading, audio, or game loops.
  - **NUMA awareness**: keep threads close to the memory bank that holds their data.
  - **Avoiding contention**: separate heavy threads onto different cores. Trade-offs: over-constraining affinity can hurt throughput if you prevent the scheduler from balancing load, or if the chosen core is throttled/parked.

Thread Binding

thread::hardware_concurrency() : how many logical CPUs we have

Affinity: ask OS scheduler to run the given thread only in the pre-defined set of CPUs.

Pthread_setaffinity_np for linux
(https://man7.org/linux/man-pages/man3/pthread_setaffinity_np.3.html)

```
cpu_set_t cpuset;
CPU_ZERO(&cpuset);
CPU_SET(i, &cpuset);
int rc = pthread_setaffinity_np(threads[i].native_handle(), sizeof(cpu_set_t), &cpuset);
```

Interesting read: https://eli.thegreenplace.net/2016/c11-threads-affinity-and-hyperthreading/

Some Links to learn about concurrent programming

https://begriffs.com/posts/2020-03-23-concurrent-programming.html

https://www.toptal.com/software/introduction-to-concurrent-programming

https://youtu.be/LOfGJcVnvAk

THANK YOU